

■ Python Complete Beginner Notes

From Zero to Exam-Ready

Covers: Variables • Loops • Lists • Dictionaries
Functions • OOP • NumPy • Matplotlib

All Practice Problems Solved | UIU Python Course

Table of Contents

Chapter 1 — Python Basics — Variables, Input, Output

Chapter 2 — Operators & Type Conversion

Chapter 3 — Conditions — if / elif / else

Chapter 4 — Loops — for & while

Chapter 5 — Strings

Chapter 6 — Lists

Chapter 7 — Tuples & Sets

Chapter 8 — Dictionaries

Chapter 9 — Functions

Chapter 10 — Classes & Object-Oriented Programming (OOP)

Chapter 11 — NumPy — Numerical Arrays

Chapter 12 — Matplotlib — Plotting Graphs

Chapter 13 — All Practice Problems — Fully Solved

What is Python?

Python is a beginner-friendly programming language. You don't need to declare variable types — Python figures it out automatically. This is called being **dynamically typed**. You run Python code line by line from top to bottom.

1.1 Variables

A variable is a name that stores a value. You create a variable simply by writing its name, an equals sign, and the value. There is no need for 'int', 'String', or any type keyword like in Java or C.

```
x = 5          # x stores the integer 5
y = 3.14       # y stores a decimal number (float)
name = 'Alice' # name stores text (string)
is_student = True # boolean: True or False

# Print them on screen
print(x)       # output: 5
print(name)    # output: Alice
print(type(x)) # output: <class 'int'>
```

✓ You can assign multiple variables in one line: a, b = 5, 10

1.2 Data Types

Type	Example	What it is
int	5, -3, 100	Whole numbers
float	3.14, -0.5	Decimal numbers
str	'hello', "hi"	Text (always in quotes)
bool	True, False	Yes/No values
list	[1, 2, 3]	Collection of items
tuple	(1, 2, 3)	Immutable collection
dict	{'a': 1}	Key-value pairs

1.3 Printing Output

Use **print()** to show something on the screen. There are several ways to format your output:

```
a = 5
b = 10
```

```
print(a)                    # 5
print(a, b)                 # 5 10
print('Value is', a)        # Value is 5

# F-string (BEST way – use this!)
print(f'{a} apples, {b} oranges') # 5 apples, 10 oranges

# Old style
print("%d apples" % a)       # 5 apples
```

1.4 Taking Input from Keyboard

Use **input()** to let the user type something. VERY IMPORTANT: `input()` ALWAYS gives you a string — even if the user types a number!

```
name = input()              # reads text as-is
n    = int(input())          # reads text, converts to integer
x    = float(input())        # reads text, converts to decimal

# You can show a prompt message:
age = int(input('Enter your age: '))
```

■ **input() returns a STRING always. If you need a number, wrap it: `int(input())` or `float(input())`. Forgetting this causes `TypeError` in your programs!**

2.1 Arithmetic Operators

```
a, b = 10, 3

print(a + b)    # 13    – addition
print(a - b)    # 7     – subtraction
print(a * b)    # 30    – multiplication
print(a / b)    # 3.33  – division (always gives float)
print(a // b)   # 3     – floor division (drops decimal)
print(a % b)    # 1     – modulus (remainder)
print(a ** b)   # 1000  – power (a to the power of b)

# Shorthand
a += 5    # same as a = a + 5
a -= 2    # same as a = a - 2
a *= 3    # same as a = a * 3
```

✓ Use ** for power. Example: 2**10 = 1024. Use % to check even/odd: n%2==0 means even.

2.2 Comparison Operators

```
x = 5
print(x == 5)    # True  – equal to
print(x != 3)    # True  – not equal
print(x > 3)     # True  – greater than
print(x < 10)    # True  – less than
print(x >= 5)    # True  – greater than or equal
print(x <= 4)    # False – less than or equal
```

2.3 Logical Operators

```
a, b = True, False
print(a and b)   # False – both must be True
print(a or b)    # True  – at least one must be True
print(not a)     # False – flips True to False

# Practical example
age = 20
is_citizen = True
if age >= 18 and is_citizen:
    print('Can vote')
```

2.4 Type Conversion (Typecasting)

You can convert between types using functions:

```
int('42')      # string to int  → 42
float('3.5')   # string to float → 3.5
str(100)       # int to string  → '100'
bool(0)        # 0 → False; anything else → True
```

Danger: adding string + number causes ERROR

x = '5' + 3 # TypeError!

x = int('5') + 3 # correct → 8

x = '5' + str(3) # string concat → '53'

■ '5' + 3 causes a **TypeError**. Always convert to the same type first!

Conditions let your program make decisions. Python uses **indentation (4 spaces or 1 tab)** to group code — there are no curly braces {}.

3.1 Basic if / elif / else

```
n = int(input())

if n > 0:
    print('Positive')
elif n < 0:
    print('Negative')
else:
    print('Zero')
```

■ Indentation is NOT optional in Python. Wrong indentation = IndentationError!

3.2 Checking Odd / Even

```
n = int(input())

if n % 2 == 0:
    print('Even')
elif n % 2 == 1:
    print('Odd')
else:
    print('ERROR')    # This runs if n is not an integer type
```

✓ `n % 2` gives the remainder when divided by 2. If remainder = 0, number is even.

3.3 Nested if (if inside if)

```
age = int(input())
is_citizen = True

if is_citizen:
    if age >= 18:
        print('Can vote')
    else:
        print('Too young to vote')
else:
    print('Not a citizen – cannot vote')
```

4.1 for loop with range()

Use a for loop when you know how many times to repeat. **range()** generates a sequence of numbers.

```
# range(stop)          → 0, 1, 2, ..., stop-1
for i in range(5):
    print(i)            # prints 0 1 2 3 4

# range(start, stop)
for i in range(1, 6):
    print(i)            # prints 1 2 3 4 5

# range(start, stop, step)
for i in range(1, 10, 2):
    print(i)            # prints 1 3 5 7 9 (odd numbers)

# Count down
for i in range(10, 0, -1):
    print(i)            # 10, 9, 8, ..., 1
```

4.2 for loop over a list or string

```
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# Iterating over a string
for char in 'Hello':
    print(char)         # H e l l o (each letter)
```

4.3 while loop

Use a while loop when you don't know in advance how many times to repeat.

```
i = 1
while i <= 5:
    print(i)
    i += 1              # MUST increment or infinite loop!

# while with condition
n = int(input())
while n != 0:
    print(n)
```

```
n = int(input()) # ask again
```

■ Always update the variable in a while loop, otherwise it runs forever (infinite loop)!

4.4 Nested loops (loop inside loop)

```
# Print all pairs from a list
l = [1, 2, 3, 4, 5]
for i in range(len(l)):
    for j in range(i+1, len(l)):
        print(l[i], l[j])

# Multiplication table
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=' ')    # end=' ' keeps on same line
    print()                     # new line after each row
```

4.5 break and continue

```
# break – stop the loop early
for i in range(10):
    if i == 5:
        break
    print(i)          # prints 0 1 2 3 4

# continue – skip this iteration
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)          # prints 1 3 5 7 9 (odd only)
```

A string is a sequence of characters. Use single quotes `' '` or double quotes `" "`. Strings are **immutable** — you cannot change individual characters.

5.1 String operations

```
s = 'Hello World'

len(s)          # 11 – number of characters
s[0]            # 'H' – first character
s[-1]           # 'd' – last character
s[0:5]          # 'Hello' – slice from index 0 to 4
s[6:]           # 'World' – from index 6 to end
s[:5]           # 'Hello' – from start to index 4
s[::-1]         # 'dlrow olleH' – reversed

s.upper()       # 'HELLO WORLD'
s.lower()       # 'hello world'
s.capitalize()  # 'Hello world' – only first letter caps
s.strip()       # remove whitespace from both ends
s.split()       # ['Hello', 'World'] – split by spaces
s.split(',')    # split by comma
'hello' in s    # True – membership check
s.replace('Hello', 'Hi') # 'Hi World'
```

5.2 Useful string tricks

```
# Join words back
words = ['I', 'love', 'Python']
' '.join(words) # 'I love Python'

# Count occurrences
s.count('l')    # 3

# Check start/end
s.startswith('He') # True
s.endswith('ld')  # True

# f-string formatting
name = 'Alice'
age = 20
print(f'My name is {name} and I am {age} years old.')
```

✓ `s.split()` is how you solve Problem 1: `'I study in UIU' → ['I','study','in','UIU']`

A list is an ordered, mutable (changeable) collection. It can hold items of any type — integers, strings, other lists, anything. Lists use square brackets `[]`.

6.1 Creating and accessing lists

```
arr = ['apple', 'grape', 100, 3.4, [1,2], (2,3)]

arr[0]          # 'apple' – first element (index starts at 0)
arr[-1]         # (2,3) – last element
arr[2]          # 100
len(arr)        # 6 – number of elements

# Check membership
100 in arr      # True
'mango' in arr  # False
```

6.2 Modifying lists

```
arr = [10, 20, 30]

arr.append(40)      # [10, 20, 30, 40] – add to end
arr.insert(1, 99)   # [10, 99, 20, 30, 40] – insert at index 1
arr.pop()           # removes last element
arr.pop(0)          # removes element at index 0
arr.remove(20)      # removes first occurrence of value 20
arr[0] = 100        # change element at index 0

arr.sort()          # sort ascending
arr.sort(reverse=True) # sort descending
arr.reverse()       # reverse the list
```

6.3 Slicing — get a part of a list

```
arr = [2, 4, 6, 8, 10, 12, 14]
      # 0  1  2  3  4  5  6  (index)

arr[1:4]          # [4, 6, 8] – index 1 to 3
arr[:3]           # [2, 4, 6] – first 3 elements
arr[4:]           # [10, 12, 14] – from index 4 to end
arr[::-1]         # [14,12,10,8,6,4,2] – reversed
arr[::2]          # [2, 6, 10, 14] – every 2nd element

# Slicing format: arr[start : stop : step]
# start is included, stop is NOT included
```

✓ `arr[-2:]` gives you the last 2 elements. `arr[1:]` removes the first element.

6.4 Copying lists — IMPORTANT!

```
foo = [1, 2, 3]

# WRONG way – both variables point to SAME list!
bar = foo
bar.append(4)
print(foo)  # [1, 2, 3, 4] ← foo also changed!

# CORRECT way – make an independent copy
foo = [1, 2, 3]
bar = foo.copy()  # or: bar = foo[:]
bar.append(4)
print(foo)  # [1, 2, 3] ← foo unchanged
print(bar)  # [1, 2, 3, 4]

# For nested lists, use deepcopy
import copy
bar = copy.deepcopy(foo)
```

■ **`bar = foo` does NOT copy — both names point to the same list in memory. Always use `.copy()`!**

6.5 Build list with a loop

```
# Take 5 integers from user
arr = []
for i in range(5):
    x = int(input())
    arr.append(x)
print(arr)
```

7.1 Tuples — immutable lists

A tuple is like a list but you CANNOT change it after creation. Tuples use parentheses ().

```
t = (1, 2, 3)
t[0]          # 1 – access works same as list
len(t)        # 3
# t[0] = 5     # ERROR! tuples cannot be changed

# Unpacking
a, b, c = t    # a=1, b=2, c=3

# Single element tuple (notice the comma!)
single = (5,) # (5) is just the number 5, not a tuple
```

✓ Use tuples when the data should not change — like coordinates (x, y) or RGB colors (255, 0, 0).

7.2 Sets — unique values only

```
s = {1, 2, 3, 2, 1} # duplicates removed automatically
print(s)           # {1, 2, 3}

s.add(4)            # add element
s.remove(2)         # remove element
3 in s              # True – membership

# Set operations
a = {1, 2, 3}
b = {2, 3, 4}
a & b               # {2, 3}      – intersection (common elements)
a | b               # {1,2,3,4}   – union (all elements)
a - b               # {1}         – difference (in a but not b)
```

A dictionary stores data as **key: value** pairs. Think of it like a real dictionary: you look up a word (key) and get its meaning (value). Dictionaries use curly braces { } with colons.

8.1 Create and access a dictionary

```
d = {
    'name'   : 'Alice',
    'age'    : 20,
    'course' : 'CSE',
    1        : 'one',      # key can be int
}

d['name']      # 'Alice' – access by key
d.get('age')   # 20      – safe access (no error if missing)
d.get('grade', 'N/A') # 'N/A' if key not found

d.keys()       # all keys
d.values()     # all values
'name' in d    # True – check if key exists
```

8.2 Modify a dictionary

```
d['age'] = 21      # update existing key
d['email'] = 'a@b.com' # add new key
del d['course']     # delete a key

# Duplicate keys: last value wins
x = {'a': 1, 'a': 99}
print(x)           # {'a': 99}
```

8.3 Iterate over a dictionary

```
for key in d:
    print(key, '-->', d[key])

# Or iterate key-value pairs together
for key, value in d.items():
    print(f'{key} = {value}')
```

■ **Keys must be immutable: str, int, tuple are OK. Lists CANNOT be dictionary keys!**

8.4 Build dictionary from two lists

```
keys   = ['a', 'b', 'c']
values = [1, 2, 3]
```

```
d = dict(zip(keys, values))
print(d)    # {'a': 1, 'b': 2, 'c': 3}

# zip() pairs up the two lists element-by-element
```

✓ This is the solution to Practice Problem 4! Remember zip() + dict().

A function is a reusable block of code. Instead of writing the same code multiple times, you define it once and call it whenever needed. Functions start with the keyword **def**.

9.1 Define and call a function

```
# Define the function
def greet(name):
    message = 'Hello ' + name
    return message          # send result back

# Call the function
result = greet('Alice')
print(result)    # Hello Alice

greet('Bob')     # Hello Bob
```

9.2 Function with no return value

```
def say_hi(name):
    print('Hi', name)    # just prints, returns nothing

say_hi('Alice')    # Hi Alice
```

9.3 Default parameter values

```
def canvote(age=0):      # age defaults to 0
    if age >= 18:
        return True
    return False

canvote(25)             # True  – age=25
canvote()               # False – age uses default 0
canvote(age=30)         # True  – keyword argument
```

■ **Default parameters must always come AFTER non-default parameters in the function definition!**

9.4 Return multiple values

```
def minmax(arr):
    return min(arr), max(arr)    # returns a tuple

result = minmax([3, 1, 9, 5])
print(result)                  # (1, 9)

# Unpack directly
```

```
mn, mx = minmax([3, 1, 9, 5])
print(mn, mx)          # 1 9
```

9.5 Check types inside a function

```
def make_dict(k, v):
    if not isinstance(k, list) or not isinstance(v, list):
        print('ERROR')
        return
    return dict(zip(k, v))

make_dict([1,2], [3,4])    # {1:3, 2:4}
make_dict('a', [1,2])     # prints ERROR
```

9.6 import math — useful math functions

```
import math

math.sqrt(25)      # 5.0  – square root
math.pi            # 3.14159...
math.floor(3.7)    # 3    – round down
math.ceil(3.2)     # 4    – round up
math.abs(-5)       # 5    – absolute value (also: abs(-5))

# Distance between two points
def distance(x1, y1, x2, y2):
    return math.sqrt((x1-x2)**2 + (y1-y2)**2)

distance(1, 1, 4, 5)  # 5.0
```

A **class** is a blueprint for creating objects. An **object** is an instance of a class — a specific thing created from the blueprint. Example: 'BankAccount' is the class; your specific account '001' is an object.

10.1 Class structure — the basics

```
class Employee:
```

```
    emp_count = 0    # CLASS variable – shared by all objects

    # Constructor – runs automatically when object is created
    def __init__(self, first, last, pay):
        self.first = first    # INSTANCE variable – unique to each object
        self.last  = last
        self.email = first + '.' + last + '@email.com'
        self.pay   = pay
        Employee.emp_count += 1

    # Method (function inside a class)
    def fullname(self):
        return f'{self.first} {self.last}'

    def applyRaise(self, raise_):
        self.pay = int(self.pay * raise_)

    # __str__ – called when you print() an object
    def __str__(self):
        return f'{self.fullname()} | {self.email} | {self.pay}'

# Create objects
emp1 = Employee('Alice', 'Smith', 50000)
emp2 = Employee('Bob', 'Jones', 60000)

print(emp1.fullname())    # Alice Smith
emp1.applyRaise(1.5)      # 50000 * 1.5 = 75000
print(emp1)               # Alice Smith | Alice.Smith@... | 75000
print(Employee.emp_count) # 2
```

■ 'self' is like 'this' in Java/C++. It refers to the current object. ALWAYS the first parameter of every method.

10.2 @staticmethod — class-level function

```

@staticmethod
def is_holiday(day):
    holidays = [1, 30, 49, 350]
    return day in holidays

Employee.is_holiday(30)    # True – call on class directly
emp1.is_holiday(30)       # also works

```

✓ Static methods don't use 'self' — they don't need a specific object. Use @staticmethod decorator.

10.3 Inheritance — child class extends parent class

```

class Developer(Employee):    # Developer inherits from Employee
    def __init__(self, first, last, pay, lang):
        super().__init__(first, last, pay) # call parent constructor
        self.lang = lang           # add new attribute

class Manager(Employee):
    def __init__(self, first, last, pay, employees=None):
        super().__init__(first, last, pay)
        self.employees = employees if employees else []

    def add_emp(self, emp):
        if emp not in self.employees:
            self.employees.append(emp)

# Check relationships
dev = Developer('Alice', 'Smith', 50000, 'Python')
isinstance(dev, Developer) # True
isinstance(dev, Employee)  # True (inherits from Employee)
issubclass(Developer, Employee) # True

```

■ Python does NOT support function overloading. If you define two functions with the same name, only the LAST one exists!

10.4 BankAccount class — Practice Problem 5

```

class BankAccount:
    def __init__(self, account_number, balance=0):
        self.account_number = account_number
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f'Deposited {amount}. New balance: {self.balance}')

    def withdraw(self, amount):
        if amount <= self.balance:

```

```
        self.balance -= amount
        print(f'Withdrew {amount}. New balance: {self.balance}')
    else:
        print('Insufficient funds!')

    def display(self):
        print(f'Account: {self.account_number}, Balance: {self.balance}')

# Test the class
acc = BankAccount('001', 1000)
acc.deposit(500)    # Deposited 500. New balance: 1500
acc.withdraw(200)   # Withdrew 200. New balance: 1300
acc.withdraw(2000)  # Insufficient funds!
acc.display()       # Account: 001, Balance: 1300
```

NumPy is a Python library for working with arrays (grids of numbers). Unlike regular Python lists, NumPy arrays can do math on all elements at once — very fast! Always import it as: **import numpy as np**

11.1 Creating arrays

```
import numpy as np

# From a list
a = np.array([1, 2, 3])          # 1D array: [1 2 3]
b = np.array([[1,2,3],[4,5,6]])  # 2D array (2 rows, 3 cols)

# Special arrays
np.zeros((3, 4))                # 3x4 array of all 0.0
np.ones((2, 2))                 # 2x2 array of all 1.0
np.full((2, 2), 5)              # 2x2 array of all 5
np.eye(3)                       # 3x3 identity matrix (1s on diagonal)

# Random arrays
np.random.random((3, 4))        # floats between 0 and 1
np.random.randint(0, 50, (4, 5)) # ints between 0 and 49
```

11.2 Shape and Reshape

```
a = np.array([[1,2,3],[4,5,6]]) # shape (2, 3)

a.shape      # (2, 3) → 2 rows, 3 columns
a.size       # 6      → total number of elements
a.dtype      # int64 → data type

# Reshape – change dimensions (total elements must stay same)
a.reshape(3, 2)  # (3, 2) – 3 rows, 2 cols
a.reshape(1, 6)  # (1, 6) – 1 row, 6 cols
a.reshape(-1)    # flatten to 1D: [1 2 3 4 5 6]

# Solution to Problem 6:
arr = np.array(arr_2d)
print(arr.reshape(1, r*c))
```

11.3 Indexing and Slicing 2D arrays

```
a = np.array([[1,2,3,4],
              [5,6,7,8],
              [9,0,1,2]])
```

```
# Single element
a[0, 2]      # row 0, col 2 → 3
a[1, 3]      # row 1, col 3 → 8

# Slicing: a[row_slice, col_slice]
a[1:3, :]    # rows 1 and 2, all cols → [[5,6,7,8],[9,0,1,2]]
a[:, 0]       # all rows, col 0 only → [1, 5, 9]
a[1:3, 2:]    # rows 1-2, cols 2+ → [[7,8],[1,2]]
a[-2:, :]     # last 2 rows → [[5,6,7,8],[9,0,1,2]]

# Slicing problem – last 2 rows and their sum:
last2 = a[-2:, :]      # or a[1:, :]
print(np.sum(last2))    # sum of all elements in last 2 rows
```

✓ 2D slicing format: `arr[row_start:row_end, col_start:col_end]`

11.4 Array Operations

```
x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])

x + y          # element-wise add → [[6,8],[10,12]]
x - y          # element-wise sub
x * y          # element-wise multiply (NOT matrix mult!)
x / y          # element-wise divide
x + 100        # add 100 to every element

# Matrix multiplication (dot product)
x.dot(y)       # or np.dot(x, y)

# Sum
np.sum(x)       # sum of ALL elements
np.sum(x, axis=0) # sum each COLUMN (collapse rows)
np.sum(x, axis=1) # sum each ROW (collapse cols)

# Transpose
x.T            # rows become columns
```

11.5 Split array

```
arr = np.array([1,2,3,4,5,6,7,8,9])
parts = np.split(arr, 3)    # split into 3 equal parts
print(parts)
# [array([1,2,3]), array([4,5,6]), array([7,8,9])]
```

■ `np.split(arr, 3)` requires the array length to be divisible by 3. Otherwise error!

Matplotlib is a library for drawing graphs and charts. Always import it as: **import matplotlib.pyplot as plt**. Always end with **plt.show()** to display the graph.

12.1 Basic line plot

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100) # 100 points from 0 to 10
y = x**2                    # y = x squared

plt.plot(x, y, label='x squared')
plt.xlabel('x')              # x-axis label
plt.ylabel('y')              # y-axis label
plt.title('My First Plot')   # title of the graph
plt.legend()                 # show the legend box
plt.grid(True)               # show grid lines
plt.show()                   # MUST call this to display!
```

12.2 Piecewise function — Practice Problem 7

```
import numpy as np
import matplotlib.pyplot as plt

# Section 1: y = x^2 for 0 <= x < 4
x1 = np.linspace(0, 4, 100)
y1 = x1**2

# Section 2: y = 16 for 4 <= x < 8
x2 = np.linspace(4, 8, 100)
y2 = np.full_like(x2, 16) # array filled with 16

# Section 3: y = (12-x)^2 for 8 <= x <= 12
x3 = np.linspace(8, 12, 100)
y3 = (12 - x3)**2

# Plot all 3 sections
plt.plot(x1, y1, label='x squared [0,4)')
plt.plot(x2, y2, label='16 [4,8)')
plt.plot(x3, y3, label='(12-x) squared [8,12]')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Piecewise Function')
```

```
plt.legend()  
plt.grid(True)  
plt.show()
```

12.3 Common plot types

```
plt.plot(x, y)           # line chart  
plt.scatter(x, y)        # scatter plot (dots)  
plt.bar(x, y)            # bar chart  
plt.hist(data, bins=10)  # histogram  
plt.pie(sizes)           # pie chart
```

✓ `np.linspace(start, stop, n)` creates `n` evenly spaced numbers between `start` and `stop` — perfect for plotting!

Problem 1: Print words separately

s.split() breaks the string at spaces and gives a list of words.

```
s = input()          # e.g. 'I study in UIU'
for word in s.split():
    print(word)

# Output:
# I
# study
# in
# UIU
```

Problem 2: Sum of squares (odd/even input)

range(1, n+1, 2) gives odd numbers: 1, 3, 5, ..., n. range(1, n+2, 2) gives odd numbers up to n+1.

```
n = int(input())
total = 0

if n % 2 == 1:          # n is odd
    for i in range(1, n+1, 2):  # 1, 3, 5, ..., n
        total += i**2
elif n % 2 == 0:       # n is even
    for i in range(1, n+2, 2):  # 1, 3, 5, ..., n+1
        total += i**2
else:
    print('ERROR')

print(total)
```

Problem 3: reverse() function

arr[::-1] is the simplest way to reverse a list in Python.

```
def reverse(lst):
    return lst[::-1]    # slice with step -1 reverses the list

arr = ['apple', 'grape', 100, 3.4, [1,2], (2,3)]
print(reverse(arr))
# Output: [(2,3), [1,2], 3.4, 100, 'grape', 'apple']
```

Problem 4: Make dictionary from keys and values lists

`isinstance(x, list)` checks if `x` is a list. `zip()` pairs up two lists. `dict()` converts pairs to dictionary.

```
def make_dict(k, v):
    if not isinstance(k, list) or not isinstance(v, list):
        print('ERROR')
        return
    return dict(zip(k, v))

keys = ['name', 'age', 'city']
values = ['Alice', 20, 'Dhaka']
print(make_dict(keys, values))
# {'name': 'Alice', 'age': 20, 'city': 'Dhaka'}

make_dict('name', [1,2]) # prints ERROR
```

Problem 5: BankAccount class

```
class BankAccount:
    def __init__(self, account_number, balance=0):
        self.account_number = account_number
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print('Insufficient funds!')

    def display(self):
        print(f'Account: {self.account_number}')
        print(f'Balance: {self.balance}')

# Test
acc = BankAccount('001', 1000)
acc.deposit(500)
acc.withdraw(200)
acc.display()
```

Problem 6: 2D NumPy array input then reshape

Build the 2D list with nested loops, convert to numpy, then reshape.

```
import numpy as np
```

```

r = int(input()) # number of rows
c = int(input()) # number of columns

arr = []
for i in range(r):
    row = []
    for j in range(c):
        row.append(int(input()))
    arr.append(row)

arr = np.array(arr) # convert list to numpy array
print(arr.reshape(1, r*c)) # reshape to (1, r*c)

```

Problem 7: Plot piecewise function (see Chapter 12 for full code)

```

# See Chapter 12 Section 12.2 for the complete code.
# Key idea: plot 3 separate (x, y) segments using plt.plot()
# x1/y1 for [0,4), x2/y2 for [4,8), x3/y3 for [8,12]
# Use np.linspace to create smooth x values for each range.

```

Problem 8: Slicing — last 2 rows of a 3x3 matrix and their sum

mat[-2:, :] means 'take the last 2 rows, all columns'.

```

import numpy as np

mat = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

last2 = mat[-2:, :] # last 2 rows (rows 1 and 2)
print('Last 2 rows:')
print(last2)
print('Sum:', np.sum(last2)) # 4+5+6+7+8+9 = 39

```

Problem 9: Split 1D array into 3 equal parts

np.split(arr, 3) divides into 3 equal parts. The array length must be divisible by 3.

```

import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90])
parts = np.split(arr, 3)

print(parts[0]) # [10 20 30]
print(parts[1]) # [40 50 60]
print(parts[2]) # [70 80 90]

```

Topic	Code	Result / Notes
Input int	<code>n = int(input())</code>	Always typecast input!
Power	<code>2**10</code>	1024
Modulus	<code>7 % 3</code>	1 (remainder)
Floor div	<code>7 // 3</code>	2
String split	<code>'a b c'.split()</code>	<code>['a','b','c']</code>
List reverse	<code>arr[::-1]</code>	Reversed list
List copy	<code>arr.copy()</code>	Independent copy
Dict access	<code>d['key']</code>	Get value
Dict safe get	<code>d.get('key', default)</code>	No error if missing
isinstance	<code>isinstance(x, list)</code>	True if x is list
zip + dict	<code>dict(zip(keys, vals))</code>	Build dict from 2 lists
numpy zeros	<code>np.zeros((3,4))</code>	3x4 array of 0s
numpy shape	<code>arr.shape</code>	(rows, cols)
numpy reshape	<code>arr.reshape(1, r*c)</code>	Flatten to 1 row
numpy 2D slice	<code>arr[1:3, 2:]</code>	Rows 1-2, cols 2+
numpy sum rows	<code>np.sum(arr, axis=1)</code>	Sum each row
numpy sum cols	<code>np.sum(arr, axis=0)</code>	Sum each col
numpy split	<code>np.split(arr, 3)</code>	Split into 3 parts
linspace	<code>np.linspace(0,10,100)</code>	100 points 0 to 10
plt show	<code>plt.show()</code>	Always at the end!

Good luck in your exam! Remember: Read carefully, write clean indented code, always typecast input(), and test with a simple example in your head.